

คู่มือส่งงานด้วย Git — ทีม Project-JhobSAMNOR

สำหรับ **ต้า** (Mobile Dev) และ **แตงกวา** (System Analyst) · repo: <https://github.com/Domezzxx/Project-JhobSAMNOR> (private) หลักการ: ห้าม push เข้า main ตรง ๆ — ทำงานบน branch ของตัวเองเสมอ แล้วเปิด PR ให้รีวิวก่อน merge

0) เตรียมครั้งแรก (ทำครั้งเดียว)

1. ขอสิทธิ์เข้า repo — repo เป็น private → ให้เบส (Domezzxx) เชิญเป็น collaborator: GitHub → repo → Settings → Collaborators → Add → ใส่ username/อีเมล แล้วกดรับ invite ในเมล
2. ติดตั้ง Git + ตั้งชื่อ/อีเมล (ครั้งเดียว): `bash git config --global user.name "ชื่อของคุณ" git config --global user.email "อีเมล GitHub ของคุณ"`
3. Clone repo มาที่เครื่อง: `bash git clone https://github.com/Domezzxx/Project-JhobSAMNOR.git cd Project-JhobSAMNOR`

1) ขั้นตอนทุกครั้งที่จะทำงาน (จำ 6 ขั้นนี้)

```
# 1. อัปเดต main ให้ล่าสุดก่อนเสมอ
git checkout main
git pull origin main

# 2. แยก branch ใหม่ของงานนี้ (ตั้งชื่อสื่อความหมาย)
git checkout -b feature/ta-goals-screen
#   ต้า:   feature/ta-<งาน>       เช่น feature/ta-budget-edit
#   แตงกวา: feature/taengkwa-<งาน>  เช่น feature/taengkwa-nav-map

# 3. ...แก้โค้ด/เขียนเอกสาร...

# 4. ดูว่าจะส่งไฟล์อะไร แล้ว stage
git status
git add .

# 5. commit (เขียนข้อความให้รู้ว่าจะทำอะไร)
git commit -m "feat(goals): หน้าตั้งเป้าออม + progress bar"

# 6. push ขึ้น branch ตัวเอง
git push -u origin feature/ta-goals-screen
```

2) เปิด Pull Request (PR) ให้รีวิว

1. หลัง push, GitHub จะขึ้นลิงก์ "Compare & pull request" — กดเปิด PR
2. ตั้ง **base = main** , **compare = branch** ของคุณ
3. เขียนสรุปสั้น ๆ: ทำอะไร, หน้าจอไหน, แบนรูป/สกรีนช็อตถ้ามี
4. ขอให้เพื่อน/เบสรีวิว → แก้ตามคอมเมนต์ (commit เพิ่มแล้ว push ซ้ำได้ PR อัปเดตเอง)
5. รีวิวผ่าน → กด **Merge** → ลบ branch ได้เลย

3) ข้อความ commit ที่ดี (convention)

รูปแบบ: ชนิด(ส่วน): สิ่งที่ทำ - feat: ฟีเจอร์ใหม่ · fix: แก้บั๊ก · docs: เอกสาร · style: ปรับ UI/จัดรูปแบบ · refactor: รื้อโค้ดไม่เปลี่ยนพฤติกรรม - ตัวอย่าง: feat(history): ฟิลเตอร์รายการตามเดือน · docs(specs): nav map หน้าแอป

4) 🚫 ห้าม commit สิ่งเหล่านี้ (มี .gitignore กันไว้แล้ว — อย่าฝืน add)

- .env / ไฟล์ที่มี API key / รหัสผ่าน (เด็ดขาด!)
- node_modules/ , build/ , *.db , โฟลเดอร์ Flutter SDK
- ถ้าเผลอ add ไฟล์ใหญ่/ลับ: `git restore --staged <ไฟล์>` เอาออกก่อน commit

5) เจอปัญหาบ่อย ๆ

Push แล้วถูกปฏิเสธ (rejected) — มีคนอัปเดต main ก่อน:

```
git checkout main && git pull origin main
git checkout feature/<ของคุณ>
git merge main          # รวมของใหม่เข้ามา
# ถ้ามี conflict: เปิดไฟล์ที่ชน แก้ส่วน <<<<<< ... ===== ... >>>>>> ให้เหลืออันที่ถูก
git add . && git commit -m "merge main"
git push
```

เผลอแก้บน main (ยังไม่ commit): ย้ายงานไป branch ใหม่

```
git checkout -b feature/<ของคุณ>  # งานที่แก้ค้างจะตามมาด้วย
```

อยากรู้ว่าตอนนี้อยู่ branch ไหน: `git branch` (มี * หน้าตัวที่อยู่)

6) Cheat Sheet

อยากทำ	คำสั่ง
ดึงของใหม่	<code>git pull origin main</code>

อยากทำ	คำสั่ง
ดูสถานะ	<code>git status</code>
แตก branch	<code>git checkout -b feature/...</code>
สลับ branch	<code>git checkout <ชื่อ></code>
บันทึก	<code>git add .</code> → <code>git commit -m "..."</code>
ส่งขึ้น	<code>git push</code>
ดูประวัติ	<code>git log --oneline -10</code>

🔑 กฎทอง: pull ก่อนเริ่ม · ทำบน branch ตัวเอง · commit บ่อย ๆ · เปิด PR ให้รีวิว · ไม่ push เข้า main ตรง